Really, CERN, why MIXMAX?

Sebastiano Vigna

Suppose you work at CERN and need a PRNG

- You open CERN's ROOT software C++ library
- You go to the section dedicated to pseudorandom number generators
- Your choices:
  - the Mersenne Twister (a classic)
  - Variants of MIXMAX (huh?)
  - Variants of RANLUX (another classic)
- And that's it
- *Hint*: literally *nobody* outside of CERN (still) uses or proposes to use this stuff

MIXMAX

- Konstantin G. Savvidy, “The MIXMAX random number generator”, Comp. Phys. Comm., 2015 (plus *so many other papers*, with *so many people named Savvidy*)
- An impressive amount of name dropping: uniformly hyperbolic Anosov C-systems, mixing of all orders, nonzero Kolmogorov entropy, chaotic trajectories, spectrum, entropy, ...
- That, apparently, justifies the unpredictability (i.e., good statistical behavior)
- The ROOT documentation says it has “proof random properties” (is that even English?)
- They even had an *event* about it! https://indico.cern.ch/event/404547/

Let's put this to test

- If it is random, it should pass statistical tests and show no bias
- The author claims it passes BigCrush (that's a pretty old test from L'Ecuyer)
- Actually, it has a number of spectral problems [L’Ecuyer, Wambergue & Bourceret, 2019], resulting in failures in *collision tests*
- Then, somebody ran PractRand (modern test) and it was failing, too
- The author “fixed” these problems by hand-tweaking some coefficients and discarding some outputs (!)
- In MIXMAX256 the situation is so bad it discards two outputs on three
- Now it passes the tests above—but maybe we are using the wrong tests

Modular rank test

- We define a modular version of the classical binary rank test on $\mathbf{F}_2$
- Fill a matrix with the output of the generator and compute its rank on $\mathbf{F}_q$
- The distribution is known [Landsberg; 1893]
- For large q, the probability of deficient rank is $\approx q^{-1}$
- More generally, the corank is d with probability $\approx (q^{-d})^d$
- Very unlikely (really strong deviation from randomness)

Modular linear-complexity test

- We define a modular version of the classical binary linear-complexity test on $\mathbf{F}_2$
- Emit a sequence of length n of elements of the generator and try to represent with a linear recurrence on $\mathbf{F}_q$ using the Berlekamp–Massey algorithm
- The complexity is $\lceil n / 2 \rceil$ with probability $1 - 1 / q$ [Gustavson; 1976]
- Every step away makes it fall by a factor q^{-2}
- Very unlikely (really strong deviation from randomness)

Let's try (any) MIXMAX

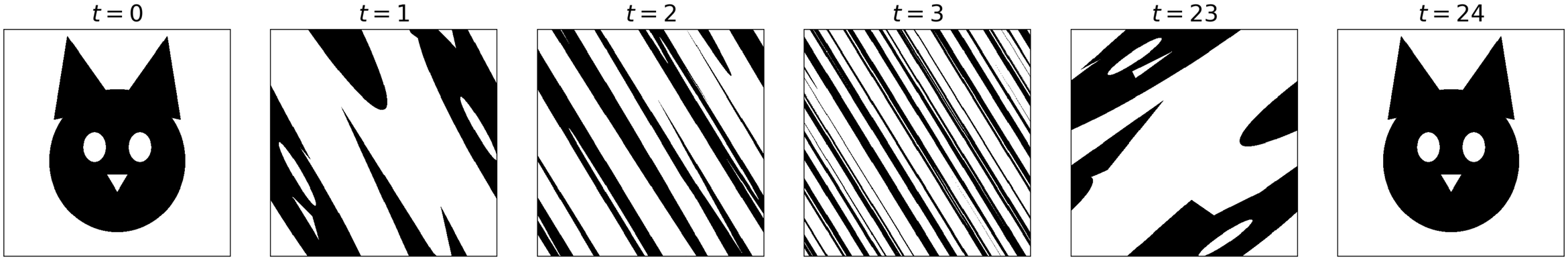
- Here $q = 2^{61} - 1$
- I developed a Rust crate `modlin` that implements these tests
- Which all MIXMAX variants fail miserably (in spite of the hand-made fixes)

```
cargo run -r -F mixmax17 -- -R 1000 -p 230584300921369395l -S 1
Generator: MIXMAX (TRandomMixMax17, N=17)
Seed: 0x0000000000000001
Running a modular rank test using 7.629 MiB of RAM: [...]
[...]
Matrix 1/1 corank=966 p=1e-307

cargo run -r -F mixmax17 -- -L 1000 -p 230584300921369395l -S 1
Generator: MIXMAX (TRandomMixMax17, N=17)
Seed: 0x0000000000000001
Running a modular linear-complexity test using 0.031 MiB of RAM: [...]
[...]
Sequence 1/1 linear complexity=272 p=1e-307
```

Why?

- MIXMAX simply multiplies a vector on $\mathbf{F}_{2^{61}-1}$ by a matrix on $\mathbf{F}_{2^{61}-1}$
- It's a linear process, obviously detectable by linear tests
- Like all linear generators, it has linear biases
- You can tweak it and fix it by hand as you like: it is a linear process and remains a linear process (last time I checked: that's not chaos)
- But wasn't it *chaotic*?
- Well... MIXMAX Anosov C-systems are linear on $\mathbf{R}^n$, but they become chaotic when you quotient on the hypercube and get a high-dimensional torus
- More importantly: the chaotic orbits *are the ones starting at irrational coordinates*



Arnold cat map on a 504 × 504 grid: every rational-lattice orbit is periodic (here, period 24)

My theory: Ethology of physicists

- *Kindchenschema* (*baby schema*): Konrad Lorenz's theory that for mammals exposure to specific traits (large head, short arms/paws) unleashes an (irrational) sense of cuteness and interest that overcomes any rational thoughts

- We are all sensitive to puppies, and, in fact, any baby-anything (lions, elephants, etc.), no matter what

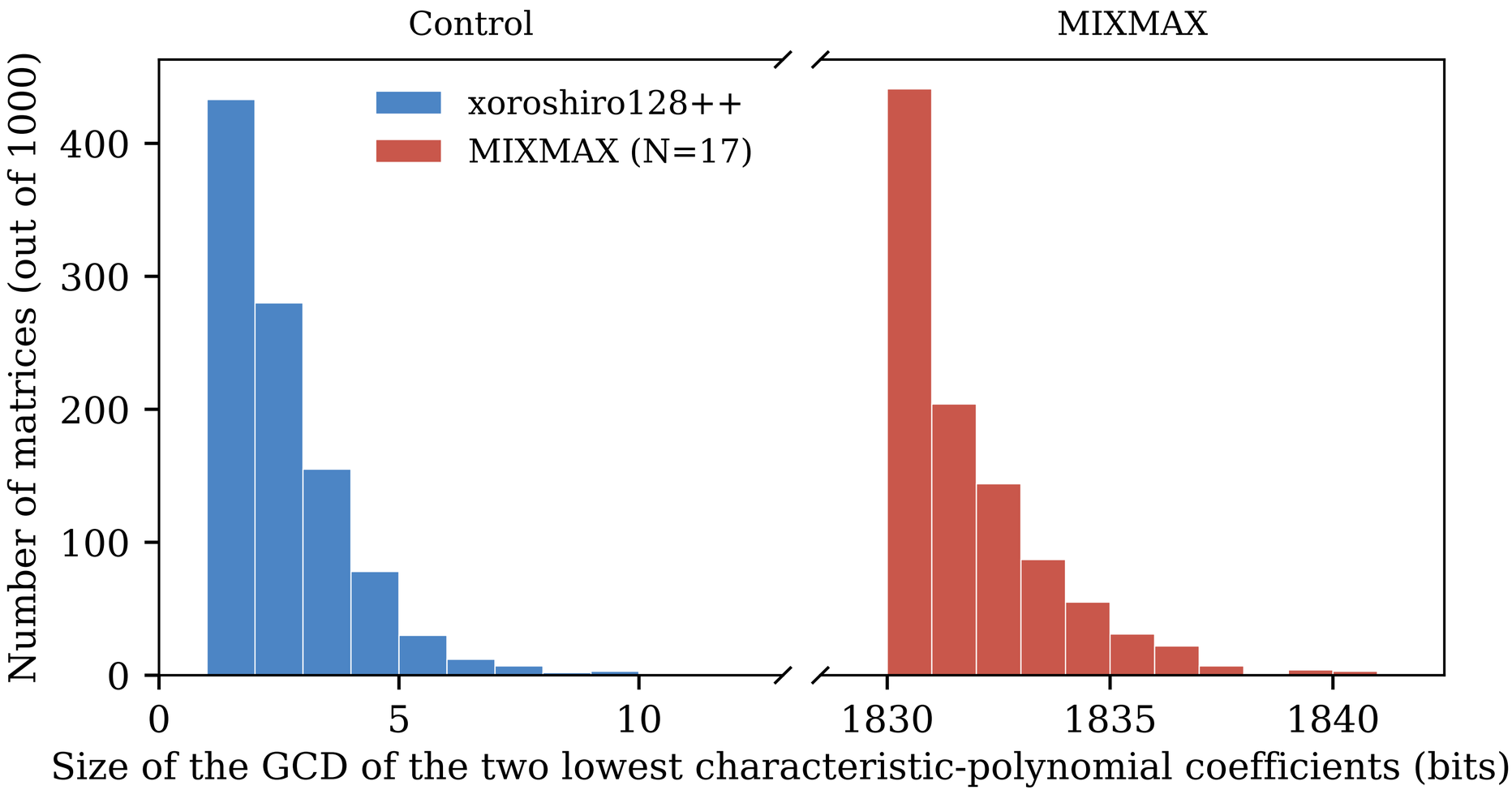
- *Chaosschema* (*chaos schema*): Sebastiano Vigna's theory that for physicists exposure to the word “chaos” unleashes an (irrational) sense of cuteness and interest that overcomes any rational thoughts

- The author of MIXMAX played his cards well with *Chaosschema*

- This is a bit like astrology or high finance

But in real life nothing bad will ever happen!

- Not really—this influences any linear computation
- Let's say I'm studying random matrices: I fill the entries with MIXMAX and compute the characteristic polynomial
- Well, everything is random, so coefficients should be more or less random—in fact, coprime
- Easy to check: let's take a couple of coefficients and check their GCD
- Let's do it for a few thousand matrices, as we can't expect to be always one
- We check the two lowest-degree coefficients
- We do the same with a control good generator (`xoroshiro128++`)


Control
MIXMAX
xoroshiro128++
MIXMAX (N=17)
Number of matrices (out of 1000)
0
100
200
300
400
0
5
10
1830
1835
1840
Size of the GCD of the two lowest characteristic-polynomial coefficients (bits)

WTF?

- Note that there is no “$2^{61} – 1$” anywhere—this is a parameterless randomness test
- But since rank is defective, the projection of the characteristic polynomial $p(x)$ on $\mathbf{F}_{2^{61}-1}$ is divisible by a significant power of x
- So the lowest coefficients are zero
- When you pull back along the projection, you get large powers of $2^{61} – 1$, hence the large GCD
- The distribution is not exact because you also get some true random GCD contribution, as in the control case

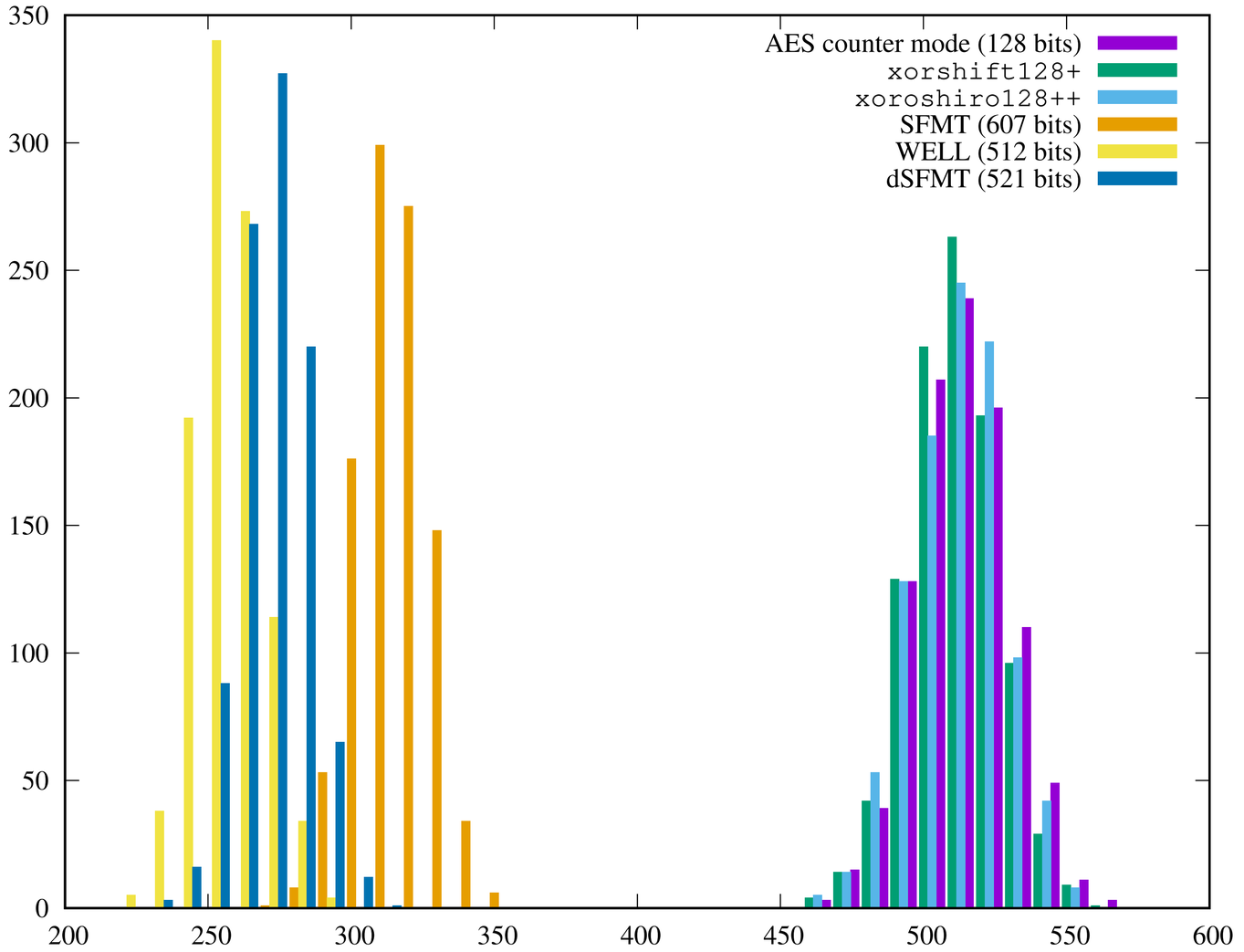
Why there wasn't any test for this

- An understandable objection would be: *you just designed this for MIXMAX*
- But, this is what happens normally: binary rank and linear-complexity tests were designed exactly when $\mathbf{F}_2$-linear PRNGs became available
- So, *why did nobody design modular tests before*?
- The answer is in a celebrated passage from Jean Kerr's “Please Don't Eat the Daisies”:

> My real problem with children is that I haven’t any imagination. I’m always warning them against the common-place defections while they are planning the bizarre and unusual. Christopher gets up ahead of the rest of us on Sunday mornings and he has long since been given a list of clear directives: ‘Don’t wake the baby,’ ‘Don’t go outside in your pajamas,’ ‘Don’t eat cookies before breakfast.’ But I never told him, ‘Don’t make flour paste and glue together all the pages of the magazine section of the Sunday Times.’ Now I tell him, of course.
>
> And then last week I had a dinner party and told the twins and Christopher not to go in the living room, not to use the guest towels in the bathroom, and not to leave the bicycles on the front step. However, I neglected to tell them not to eat the daisies on the dining-room table. This was a serious omission, as I discovered when I came upon my centerpiece—a charming three-point arrangement of green stems.

The Mersenne Twister

- Let's have a look at another generator sponsored by CERN: the Mersenne Twister [Matsumoto & Nishimura; 1998]
- You might be aware it is again a linear generator, this time on $\mathbf{F}_2$
- Here things can get *really spooky*
- Let's say we fill a matrix with nonuniform probabilities: Pr[2] = 1/4, Pr[5] = 1/2, Pr[8] = 1/4.
- We compute the characteristic polynomial, which should be reasonably random
- We check the number of odd coefficients, just to be sure, against a few other control generators

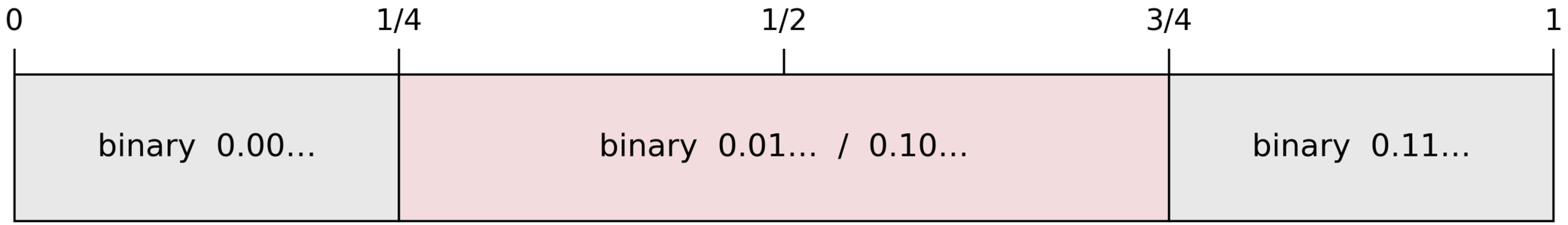


Number of odd coefficients in the characteristic polynomial of 1024 × 1024 pseudorandom integer matrices with entries satisfying Pr[2] = 1/4, Pr[5] = 1/2, Pr[8] = 1/4

WTF²?

- Little quiz: how is this possible?
- [Pause and discussion]
- The sneaky trick: floating point numbers are obtained dividing the output of a generator by a power of 2 (or directly, as in the dSFMT)
- In IEEE 754 representation, the two most significant fractional digits of numbers in [0..1/4) (→2), are 00, those of [1/4..3/4) (→5) are 01 or 10, and those of [3/4..1) (→8) are 11
- This means that *the parity of a coefficient depends on the XOR of the two most significant fractional digits*
- Every bit of an $\mathbf{F}_2$-linear generator obeys the same $\mathbf{F}_2$-linear recurrence, and *the XOR of two bits obeying the same recurrence follows again the same recurrence*

uniform float in [0..1): the two most significant fraction bits $b_1 b_2$



parity of the entry = $b_1 \oplus b_2$, an $\mathbf{F}_2$-linear function of the generator's state

CERN, really?

- You suggest using 3 generators
- 2 of them have pretty horrible linear statistical biases
- And “it provides the capability to be seeded with the guarantee that, for each given different seed, a different sequence of random numbers will be generated”—well, *I really hope so*! (I actually think they mean “nonoverlapping in the orbit”, but, still...)
- The last one, RANLUX, is a dinosaur from times when generators were so bad that they were *decimated* to get a decent output
- ...but https://luscher.web.cern.ch/luscher/ranlux/
- *Chaosschema*!

“Given Different Seeds”?

- Let's get back to “it provides the capability to be seeded with the guarantee that, for each given different seed, a different sequence of random numbers will be generated”
- This is *statistical nonsense*, AKA *statistical soup of words*
- The generator is *linear*: when you have two states ***s*** and ***t***, there is an invertible matrix $S = M^k$ for some k such that $\boldsymbol{s} = \boldsymbol{t}S$, $\boldsymbol{s}M = \boldsymbol{t}SM$, etc.
- So, no matter how you stochastically model the output sequence starting from ***s*** as a process, the sequence starting from ***t*** is in bijection
- Statistically, they are the same process—there can be no statistical independence: simply, *S* is complicated enough *that you (and statistical tests) do not notice*
- (This is true of any “structured” generator)

Please use stream ciphers

- I've been asked by physicists in simulation “do you think this makes sense”?
- My answer is: no, it does not make any sense—it's just horribly embarrassing to think that an institution using billions of euros to build kilometer-long rings of technology to break matter is using flawed generators
- In 2026, if you need a generator and you do not have specific constraints, *use a stream cipher*
- ChaCha20, AES in counter mode, Philox, whatever [Hellekalek & Wegenkittl, 2003; Salmon, Moraes, Dror & Shaw, 2011]
- AES in counter mode: encode a 128-bit counter with AES; the key is the seed
- On modern hardware, AES in counter mode ≈1ns for 64 random bits

Stream ciphers are really (pseudo)random

- Stream ciphers generate pseudorandom bits that you XOR with your message
- How random?
- The smartest people with basically infinite budget have been unable to find any statistical bias
- This is a better guarantee than *any* specific randomness test
- The only alternative is *provably* random generators, and we're very far from getting anything usable [Impagliazzo, Levin & Luby, 1989], as we need *at least* $P \neq NP$
- If you have special needs (real-time, very low memory, simplified instructions, etc.) shop for a fast generator that fits your needs and passes all known tests

Fight *Chaosschema*!

Please Use Stream Ciphers